DUALE HOCHSCHULE SACHSEN

Dr. M. Hofmann

Verteilte Systeme

Wintersemester 2026

2. Übungsserie

Aufgabe 1

Machen Sie sich mit dem Robot Operating System (ROS) Version 2 vertraut, indem Sie das nachfolgende Producer-Consumer-Demonstrationsbeispiel ausführen.
Hinweis: Installieren Sie ROS 2 unter Windows bzw. Ubuntu Linux (nativ oder WSL) mit den Anweisungen am Ende dieses Übungsblattes.
Datei: `producer.py`

```python
#!/usr/bin/env python

import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node
from std_msgs.msg import String

class ProducerNode(Node):
    def __init__(self):
        super().__init__("producer_node")
        self._pub = self.create_publisher(String, "chatter", 10)
        self._count = 0
        timer_period = 1.0  # seconds (1 Hz)
        self._timer = self.create_timer(timer_period, self._callback)

    def _callback(self):
        msg = String()
        msg.data = f"Hello ROS {self._count}"
        self.get_logger().info(f"I send '{msg.data}'")
        self._pub.publish(msg)
        self._count += 1

if __name__ == "__main__":
    try:
        with rclpy.init():
            node = ProducerNode()
            rclpy.spin(node)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
```

Datei: `consumer.py`

```python
#!/usr/bin/env python3

import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node
from std_msgs.msg import String

class ConsumerNode(Node):
    def __init__(self):
        super().__init__('consumer_node')
        self._sub = self.create_subscription(String, "chatter", self._callback,
                                             10)

    def _callback(self, msg):
        self.get_logger().info(f"I heard '{msg.data}'")

if __name__ == "__main__":
    try:
        with rclpy.init():
            node = ConsumerNode()
            rclpy.spin(node)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
```

Aufgabe 2

Mehrere ROS-Knoten sollen Sensordaten (z. B. Temperatur, Position oder Statusmeldungen) generieren und an einen zentralen Aggregator-Knoten senden. Der Zentralknoten verarbeitet die Daten und gibt sie in konsolidierter Form aus (z. B. in der Konsole oder über `rviz`/`rqt_plot`).

a) Erstellen von Sensorknoten (Publisher):

- Implementiere mindestens drei verschiedene Publisher-Knoten.
- Jeder Publisher veröffentlicht periodisch Daten, z. B.:

  Knoten 1 Temperaturwerte (Float)

  Knoten 2 Roboterstatus (String, z. B. „Moving“ / „Idle“)

  Knoten 3 Zufallsposition (x, y als `geometry_msgs/Point`)
- Nutze unterschiedliche Topics für jeden Publisher (/temperature, /status, /position).

b) Zentraler Aggregator-Knoten (Subscriber):

- Implementiere einen Subscriber-Knoten, der alle drei Topics abonniert.
- Dieser Knoten soll die eingehenden Daten dynamisch zusammenführen.
- Darstellung in der Konsole als „Snapshot“ aller letzten bekannten Werte, zum Beispiel:

```
[Aggregator]
Temperature: 22.5 °C
Status: Moving
Position: (1.2, -0.8)
```

c) Erweiterung:

- Baue eine Logik ein, dass der Aggregator bei Änderung der Daten farbliche Markierungen oder Warnungen einblendet (z. B. Temperatur > 30 °C $\rightarrow$ Warnung).
- Nutze `rqt_plot` oder `rqt_gui`, um die Werte visuell darzustellen.

Installation von ROS 2 unter Windows

1. Erstellen des Workspace-Verzeichnis, z. B. `c:\pixi_ws` oder `%USERPROFILE%\pixi_ws`
2. Installation des Paketmanagers Pixi:
   - Standalone-Version herunterladen: `https://github.com/prefix-dev/pixi/releases/download/v0.55.0/pixi-x86_64-pc-windows-msvc.zip`
   - Entpacken und Datei `pixi.exe` im Workspace-Verzeichnis ablegen.
3. Installation von Abhängigkeiten:
   - Datei `pixi.toml` herunterladen und im Workspace-Verzeichnis speichern: `https://raw.githubusercontent.com/ros2/ros2/refs/heads/kilted/pixi.toml`
   - Eingabeaufforderung (`cmd`) im Workspace-Verzeichnis öffnen.
   - Abhängigkeiten installieren: `pixi install`
4. Installation von ROS 2, Version: *Kilted Kaiju*:
   - Release herunterladen: `https://github.com/ros2/ros2/releases/download/release-kilted-202ros2-kilted-20250728-windows-release-amd64.zip`
   - Entpacken und Ordner `ros2-windows` im Workspace-Verzeichnis ablegen.
   - Pfad zur Datei `python.exe` in allen ROS 2 Dateien anpassen, z. B. Workspace-Verzeichnis mit VSCode öffnen, globales Suchen-und-Ersetzen (Shift+Strg+H) von `c:\pixi_ws` durch Workspace-Verzeichnis.
5. Testen von ROS 2:
   - Eingabeaufforderung (`cmd`) im Workspace-Verzeichnis öffnen.
   - Pixi-Umgebung einrichten: `pixi shell`
   - ROS 2-Umgebung einrichten: `call ros2-windows\local_setup.bat`
   - ROS 2-Kommando ausführen: `ros2`
   - Demo-Beispiel ausführen:
     - Publisher-Knoten: `ros2 run demo_nodes_py talker`
     - Subscriber-Knoten: `ros2 run demo_nodes_py listener` (separate `cmd`)

Installation von ROS 2 unter Ubuntu 24.04 Linux

1. Installation von ROS 2, Version: *Kilted Kaiju*:
   - Source-Paket herunterladen: `https://github.com/ros-infrastructure/ros-apt-source/releases/download/1.1.0/ros2-apt-source_1.1.0.noble_all.deb`
   - Source-Paket installieren: `sudo dpkg -i ros2-apt-source_1.1.0.noble_all.deb`
   - ROS 2 Basis-Paket und Python-Demos installieren: `sudo apt install ros-kilted-ros-base ros-kilted-demo-nodes-py`
2. Testen von ROS 2:
   - Kommandozeile öffnen.
   - ROS 2-Umgebung einrichten: `source /opt/ros/kilted/setup.bash`
   - ROS 2-Kommando ausführen: `ros2`
   - Demo-Beispiel ausführen:
     - Publisher-Knoten: `ros2 run demo_nodes_py talker`
     - Subscriber-Knoten: `ros2 run demo_nodes_py listener` (separate `cmd`)